

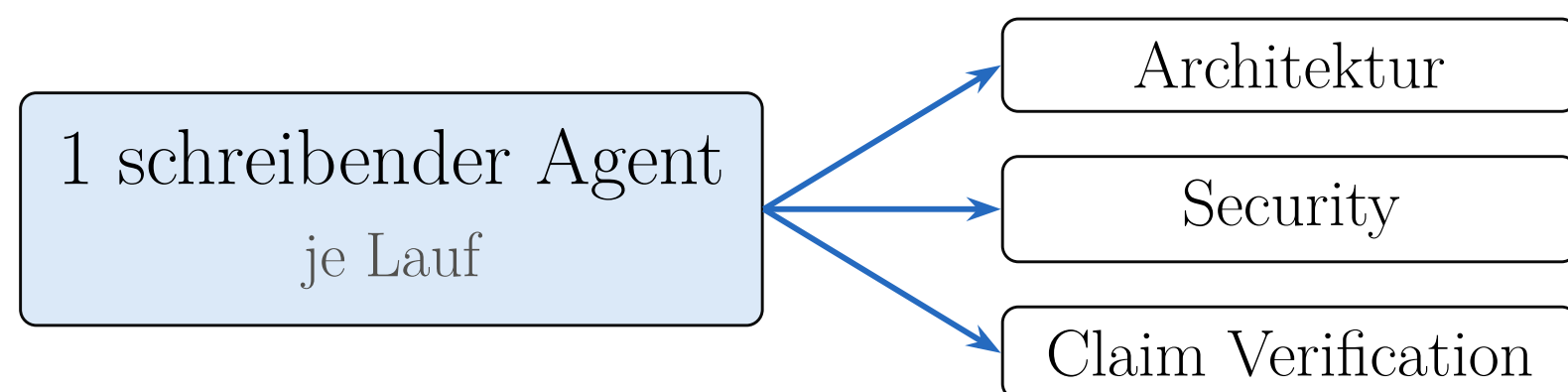
# Eine Architektur aufschreiben ist das eine. Sie zu bauen korrigiert sie.

Aus einem Whitepaper über den kontrollierten Einsatz von Coding-Agenten wurde ein laufendes System. Was dabei anders kam als geplant — vier Positionsverschiebungen aus der Praxis.

# 28 fachliche Slices, 29 Releases, vier Monate.

Rund 36.600 Zeilen Produktivcode und 269 Testklassen; die Testabdeckung bricht den Build, wenn sie unter die Schwelle fällt (Line 85 %, Branch 81 %). Java 25, Spring Boot 4, ein Deployment-Artefakt. Stand: 7. August 2026, Release 0.22.0.

# Perspektivenvielfalt ist beim Prüfen wertvoller als beim Erzeugen.



Geplant waren sieben parallele Spezialagenten. Gebaut wurde ein schreibender Agent je Lauf — und mehrere unabhängige Prüfer danach. Die Parallelität wanderte von der Erzeugung auf die Bewertung.

# Ein Retry ohne neue Information wiederholt nur den Fehler.

Der Agent scheitert seltener am Programmieren als an der Realität des Repositories: veralteter Base-Branch, fremde parallele Änderungen, fremde CI. Wer das nicht als Aufgabe modelliert, hört dort auf, wo die interessante Arbeit anfängt.

# Die Nachweisstruktur lässt sich nicht sauber nachrüsten.

Acht aufeinanderfolgende Datenbankmigrationen enthielten nichts als Mandanten- und Nachweisstrukturen — und bestimmten rückwirkend, an welchen Stellen im Ablauf überhaupt Ereignisse entstehen müssen. Wer sie früher entwirft, spart die Nacharbeit.



# Kubernetes ist keine Voraussetzung. Souveränität schon.

Geplant war eine Kubernetes-Deployment-Architektur. Gefragt wurde zuerst nach Betreibbarkeit im eigenen Haus: ein Anwendungscontainer, eine Datenbank, lokale Modelle, bis hin zum Air-Gap-Betrieb. Skalierende Infrastruktur ist Option, nicht Voraussetzung.

# Ein Gate, das sich selbst überspringt, sieht aus wie ein bestandenes Gate.

Der Abhängigkeits-Scan der eigenen CI lief monatelang ohne gültigen Zugang zur Schwachstellendatenbank — und meldete durchgehend Grün, weil er zusätzlich als nicht blockierend konfiguriert war. Ein Prüfschritt muss nicht nur existieren; er muss beweisen, dass er gelaufen ist.

**Ein System, das seine  
Architekturschuld zählt, ist der  
bessere Beleg als eines, das  
angeblich keine hat.**

Das Kapitel benennt zehn offene Punkte, eingefrorene Modulzyklen und einen Testdefekt. Und es behauptet ausdrücklich keine Produktivitäts- oder ROI-Zahlen: Dafür fehlt die kontrollierte Messung.



# Erst die Koordination beweisen. Dann die Schreibrechte verteilen.

Seit Release 0.22.0 schreiben mehrere Agenten parallel — in getrennten Arbeitsverzeichnissen, koordiniert über einen Abhängigkeitsgraphen. Wessen Schreibbereiche sich mit einem aktiven Task überschneiden, startet gar nicht erst; Build-Dateien und Migrationen sind immer exklusiv. Eine Merge Queue führt die Branches in Planreihenfolge zusammen, ein Integration Gate prüft den Gesamtstand. Alles hinter einem standardmäßig deaktivierten Feature-Flag.

# Die SoftwareFabrik

Von der Referenzarchitektur zum System — Aufbau, Governance  
und Grenzen einer Referenzimplementierung

**Sonderdruck und vollständiges Whitepaper: Download-Link  
im Beitrag**

janda.io · 33 Seiten, Stand: August 2026. Alle Angaben aus dem  
Quellcode erhoben.